Minor Assignment 1 - CS771

Golla Lekha Harshaa
CSE / 240402
lekhahg24@iitk.ac.in

Korada Jayavardhan
CSE / 240554
jkorada24@iitk.ac.in

Yash Raj
CSE / 241202
yashr24@iitk.ac.in

Panchaneni Ashwin Rao
CSE / 240727
ashwinr24@iitk.ac.in

Vandana
Statistics / 252080612
vandana25@iitk.ac.in

Abstract

This report details our findings for Minor Assignment 1 (Testing the Limits). We investigate the effects of various SVM hyperparameters using synthetic 2D datasets across different challenge levels, analyzing accuracy, decision boundaries, and training times.

Problem 1.1: Testing the Limits

Part 1: Linear Separability of Synthetic Data

Answer: Yes. The data generated by the `genChallengeData` function will always be linearly separable for all valid values of `level` (`level` > 0) and $n \geq 0$.

Mathematical Proof (Geometric Approach)

To prove that the generated data is always linearly separable, we must demonstrate that there always exists a 2D hyperplane (a straight line) that perfectly divides the positive class (`XPos`) from the negative class (`XNeg`) with a margin strictly greater than zero.

1. Definition of Parameters and Centers Let L denote the `level` parameter. The problem dictates that $L > 0$. By inspecting the `genChallengeData` function, the data points are sampled from circles of radius $r = 1$. Let us define a constant α based on the code's configuration:

$$\alpha = \sqrt{2}\left(1 + \frac{1}{2L}\right)$$

Because $L > 0$, the term $\left(1 + \frac{1}{2L}\right)$ is strictly greater than 1. Therefore, it is mathematically guaranteed that $\alpha > \sqrt{2}$.

The data classes are centered at the following coordinates:

- Positive Class Circle 1 (C_{P1}): $(-\alpha, \sqrt{2})$
- Positive Class Circle 2 (C_{P2}): $(\alpha, -\sqrt{2})$
- Negative Class Circle (C_N): $(-\alpha, -\sqrt{2})$

All generated points lie exactly on the boundary of circles of radius $r = 1$ centered at these coordinates.

Preprint.

2. Geometric Construction of the Separating Hyperplane Consider the triangle formed by connecting the three centers: $\triangle C_N C_{P1} C_{P2}$.

- The leg $C_N C_{P1}$ lies along the vertical line $x = -\alpha$. Its length is $\sqrt{2} - (-\sqrt{2}) = 2\sqrt{2}$.
- The leg $C_N C_{P2}$ lies along the horizontal line $y = -\sqrt{2}$. Its length is $\alpha - (-\alpha) = 2\alpha$.

Because one leg is strictly vertical and the other is strictly horizontal, the angle at vertex C_N is exactly $90°$. Thus, $\triangle C_N C_{P1} C_{P2}$ is a right-angled triangle.

The positive class points are clustered around C_{P1} and C_{P2}. The optimal linear decision boundary separating the positive class from the negative class will be parallel to the hypotenuse connecting C_{P1} and C_{P2}.

3. Defining the Margin Using Tangents Since the positive circles have the identical radius $r_{pos} = 1$, their external common tangent, denoted as T_p, is parallel to the hypotenuse $C_{P1}C_{P2}$. Similarly, the tangent to the negative circle (radius $r_{neg} = 1$) that is closest to the positive class and parallel to the hypotenuse is denoted as T_n.

The perpendicular distance from the negative center C_N to the hypotenuse $C_{P1}C_{P2}$ is exactly the geometric altitude h of the right-angled triangle. Since the tangents T_p and T_n are offset from their respective centers by the radius of the circles ($r = 1$), the separating margin (the perpendicular distance between the classes) is:

$$\text{Margin} = h - r_{pos} - r_{neg} = h - 1 - 1 = h - 2$$

For the data to be strictly linearly separable with 100% accuracy, this margin must be strictly positive. Therefore, we must prove that $h > 2$.

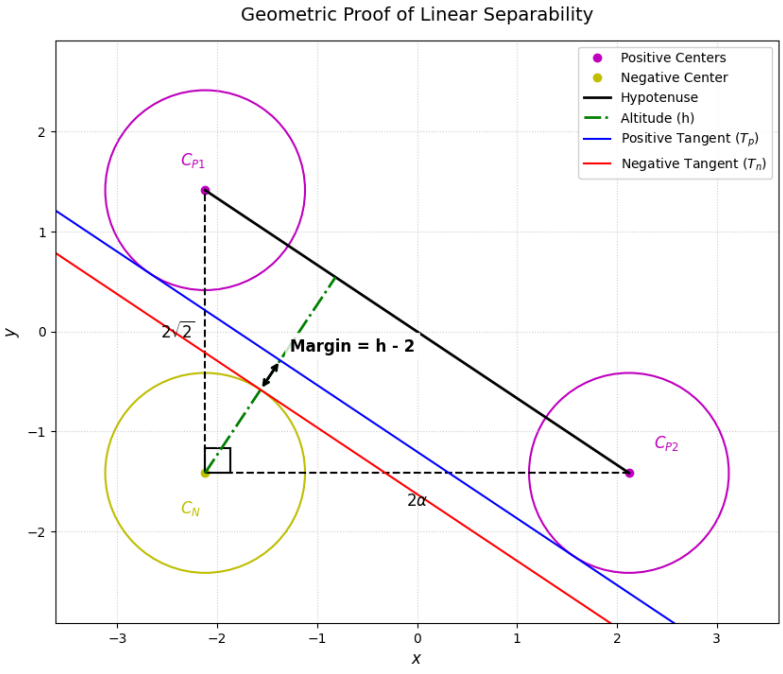

Figure 1: Geometric analysis of class distributions: the distance between the positive common tangent T_p and the negative tangent T_n defines the strict separability of the 2D synthetic dataset.

4. Calculation of Altitude h We can compute the area of $\triangle C_N C_{P1} C_{P2}$ in two different ways. First, using the perpendicular legs:

$$\text{Area} = \frac{1}{2} \times \text{base} \times \text{height} = \frac{1}{2}(2\alpha)(2\sqrt{2}) = 2\alpha\sqrt{2}$$

Second, calculating the length of the hypotenuse using the Pythagorean theorem:

$$|C_{P1}C_{P2}| = \sqrt{(2\alpha)^2 + (2\sqrt{2})^2} = \sqrt{4\alpha^2 + 8} = 2\sqrt{\alpha^2 + 2}$$

Using the hypotenuse and the corresponding altitude h, the area is:

$$\text{Area} = \frac{1}{2} \times \text{hypotenuse} \times h = \frac{1}{2}\left(2\sqrt{\alpha^2 + 2}\right) h = h\sqrt{\alpha^2 + 2}$$

Equating the two area expressions yields the value of h:

$$h\sqrt{\alpha^2 + 2} = 2\alpha\sqrt{2} \Rightarrow h = \frac{2\alpha\sqrt{2}}{\sqrt{\alpha^2 + 2}}$$

5. Proving $h > 2$ We substitute the calculated value of h into our required inequality $h > 2$:

$$\frac{2\alpha\sqrt{2}}{\sqrt{\alpha^2 + 2}} > 2$$

Dividing both sides by 2 gives:

$$\frac{\alpha\sqrt{2}}{\sqrt{\alpha^2 + 2}} > 1$$

Because $\alpha > 0$, both sides of the inequality are strictly positive, allowing us to square both sides while preserving the inequality direction:

$$\frac{2\alpha^2}{\alpha^2 + 2} > 1$$

Multiplying both sides by the positive denominator $(\alpha^2 + 2)$:

$$2\alpha^2 > \alpha^2 + 2$$

Subtracting α^2 from both sides yields:

$$\alpha^2 > 2$$

As established in step 1, the initialization logic strictly enforces that $\alpha > \sqrt{2}$ for any `level` > 0. Consequently, the inequality $\alpha^2 > 2$ is an absolute mathematical certainty.

Conclusion Because $\alpha^2 > 2$ is always true, the altitude h is strictly greater than 2. Therefore, the distance between the closest boundaries of the positive and negative distributions $(h - 2)$ is strictly greater than zero. A linear hyperplane can perpetually be drawn through this gap, definitively proving that the generated data is linearly separable for all valid parameters.

Part 2: Hyperparameter Tuning for Small Challenge Levels

For small challenge levels ($level < 0.5$), the objective is to experimentally find the smallest value for the hyperparameters `myC` and `max_iter`, and the largest value of the hyperparameter `myTol`, for which the SVM solver still yields 100% accuracy. To visualize this, we used a 2D problem to understand the effects of various SVM hyperparameters. We generated synthetic data using the `genChallengeData` function with the arguments $level = 0.2$ and $n = 50$.

1. Hyperparameter Exploration

To systematically find the absolute limits of the LinearSVC model, we tested various combinations of the regularization parameter (`myC`), maximum iterations (`max_iter`), and tolerance (`myTol`). Our experimental methodology was broken down into a step-by-step isolation process, documented in the tables below.

Step 1: Minimizing Regularization (`myC`) First, we kept `max_iter` at a high baseline (1000) and `myTol` at a strict baseline (0.0001) while gradually decreasing `myC`.

Table 1: Minimizing `myC` with constant iterations and tolerance

myC	max_iter	myTol	Acc
1	1000	0.0001	1
0.1	1000	0.0001	1
0.01	1000	0.0001	1
0.001	1000	0.0001	0.82
0.005	1000	0.0001	0.93
0.008	1000	0.0001	0.97
0.009	1000	0.0001	0.98
0.0095	1000	0.0001	0.99
0.0098	1000	0.0001	0.99
0.0099	1000	0.0001	1
0.00985	1000	0.0001	1
0.00983	1000	0.0001	1
0.00981	**1000**	**0.0001**	**1**

Finding: The least value of `myC` for which accuracy remains at 100% is **0.00981**.

Step 2: Minimizing Iterations (`max_iter`) at Strict Tolerance Next, using `myC = 0.00981` and maintaining the strict `myTol = 0.0001`, we reduced the maximum number of iterations.

Table 2: Minimizing `max_iter` at strict tolerance

myC	max_iter	myTol	Acc	Notes
0.00981	100	0.0001	1	
0.00981	10	0.0001	1	
0.00981	**4**	**0.0001**	**1**	**Safe Minimum**
0.00981	3	0.0001	1	*ConvergenceWarning generated*

Finding: At `max_iter = 3`, the model triggered a `ConvergenceWarning: Liblinear failed to converge`. Therefore, under a strict tolerance, the minimum viable `max_iter` is **4**.

Step 3: Maximizing Tolerance (`myTol`) Using our established baselines of `myC = 0.00981` and `max_iter = 4`, we increased the tolerance until the model's accuracy dropped.

Table 3: Maximizing `myTol` limits

`myC`	`max_iter`	`myTol`	Acc	Notes
0.00981	4	0.001	1	
0.00981	4	0.01	1	
0.00981	4	0.1	1	
0.00981	4	1	1	
0.00981	4	2	1	
0.00981	4	2.5	1	
0.00981	4	2.8	1	
0.00981	4	2.9	1	
0.00981	4	2.95	1	
0.00981	4	2.98	1	
0.00981	4	2.99	1	
0.00981	**4**	**2.999**	**1**	
0.00981	4	3.0	0.33	*(Threshold crossed)*

Finding: The largest possible tolerance that maintains 100% accuracy is **2.999**. At higher values (e.g., 3.0, 5, 10), accuracy drops sharply to 0.33.

Step 4: Re-evaluating `max_iter` with Maximum Tolerance Because maximizing `myTol` inherently relaxes the stopping criteria, we hypothesized that the convergence error previously seen at low iterations might resolve. We tested `max_iter` again using the new optimal `myTol = 2.999`.

Table 4: Re-evaluating `max_iter` with relaxed tolerance

`myC`	`max_iter`	`myTol`	Acc	Notes
0.00981	4	2.999	1	No warning
0.00981	3	2.999	1	No warning
0.00981	**2**	**2.999**	**1**	**New Safe Minimum**
0.00981	1	2.999	1	*ConvergenceWarning returns*

Finding: When applying the maximum tolerance, the `ConvergenceWarning` that previously appeared at `max_iter = 3` disappeared. We were able to push the iterations down to **2** before the error returned at `max_iter = 1`.

2. Optimal Hyperparameters Found

Because the data points are placed far away from the margin at $level = 0.2$, the model requires very few iterations and minimal regularization to find a successful separating hyperplane. Our progressive experiments show the true absolute limits that still yield 100% accuracy without throwing convergence warnings are:

- **Smallest** `myC`: 0.00981
- **Smallest** `max_iter`: 2
- **Largest** `myTol`: 2.999

3. Decision Boundary Visualization

Figure 2 shows the decision boundary plotted using the `shade2D` and `plot2D` functions. The background shading represents the separating hyperplane, accurately dividing the positive and negative classes despite extreme regularization, a high tolerance for stopping, and very early termination.

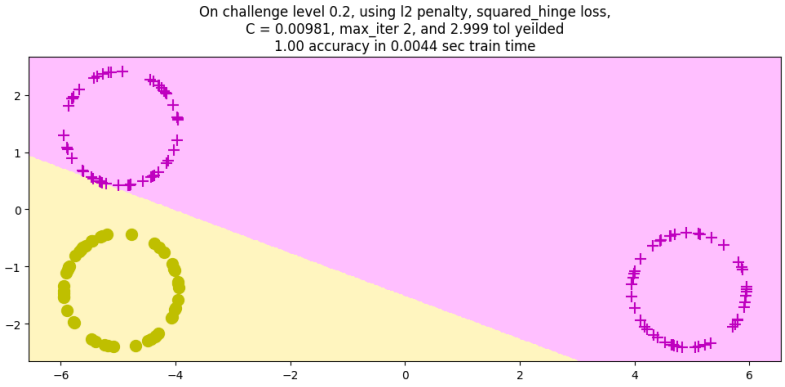

Figure 2: Decision boundary for $level = 0.2$ using `l2` penalty, `squared_hinge` loss, $C = 0.00981$, $max\_iter = 2$, and $tol = 2.999$.

Note on Data Variance: It is important to note that for a fixed sample size ($n = 50$), regenerating the synthetic data points will slightly alter the exact convergence thresholds. Consequently, the absolute minimums for `myC` and `max_iter`, as well as the absolute maximum for `myTol` reported in this section, are specific to this particular realization of the dataset. However, because the classes are well-separated at $level = 0.2$, the variation in these optimal hyperparameters across different data generations remains extremely small.

Part 3: Hyperparameter Tuning for Large Challenge Levels

For large challenge levels ($level > 50$), the objective is to experimentally find the smallest value for the hyperparameters `myC` and `max_iter`, and the largest value of the hyperparameter `myTol`, for which the SVM solver still yields 100% accuracy. We again consider a 2D problem to visualize the effects of the SVM hyperparameters. Synthetic data was generated using the `genChallengeData` function with arguments $level = 60$ and $n = 50$.

1. Hyperparameter Exploration
To demonstrate our work, we evaluated the accuracy for several combinations of the hyperparameters. We varied the regularization parameter (`myC`), the maximum number of iterations (`max_iter`), and the tolerance parameter (`myTol`). The results of our experiments are summarized in Table 5.

Table 5: Hyperparameter exploration for $level = 60$, $n = 50$

myC	max_iter	myTol	Acc
1000	100	10^{-4}	1.00
900	100	10^{-4}	0.99
950	100	10^{-4}	1.00
925	100	10^{-4}	1.00
912	100	10^{-4}	0.99
913	100	10^{-4}	0.99
920	100	10^{-4}	1.00
919	100	10^{-4}	1.00
918	100	10^{-4}	0.99
918.5	100	10^{-4}	1.00
918.4	100	10^{-4}	1.00
918.36	100	10^{-4}	1.00
918.35	100	10^{-4}	0.99
918.352	100	10^{-4}	0.99
918.353	100	10^{-4}	1.00
918.353	50	10^{-4}	0.99
918.353	75	10^{-4}	0.99
918.353	85	10^{-4}	0.99
918.353	95	10^{-4}	0.99
918.353	17	10^{-4}	1.00
918.353	16	10^{-4}	0.99
918.353	17	10^{-3}	0.99
918.353	17	10^{-4}	1.00
918.353	17	1.6×10^{-4}	1.00
918.353	17	1.63×10^{-4}	1.00
918.353	17	1.6301×10^{-4}	1.00
918.353	17	1.6302×10^{-4}	0.99

2. Optimal Hyperparameters Found
For challenge level 60, the dataset becomes harder to separate compared to the small challenge level case. After extensive experimentation, the smallest and largest hyperparameter values that still achieve 100% accuracy are:

- **Smallest** `myC`: 918.353
- **Smallest** `max_iter`: 17
- **Largest** `myTol`: 1.6301×10^{-4}

3. Decision Boundary Visualization
Figure 3 shows the decision boundary produced by the SVM classifier using the optimal hyperparameters. The shaded background produced by `shade2D` indicates the separating hyperplane,

while `plot2D` shows the positive and negative samples. Even with a high challenge level, the model successfully identifies a separating hyperplane.

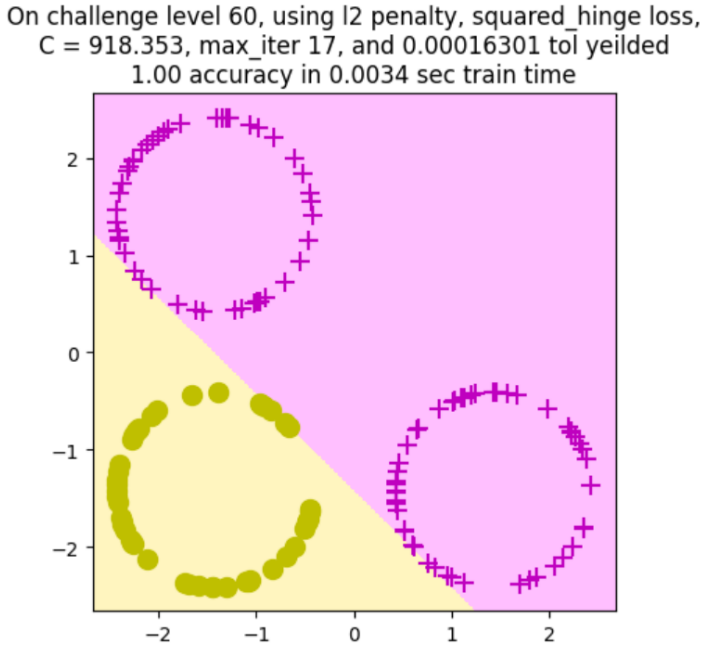

Figure 3: Decision boundary for $level = 60$ using `l2` penalty, `squared_hinge` loss, $C = 918.353$, $max\_iter = 17$, and $tol = 1.6301 \times 10^{-4}$.

Note on Data Variance: It is important to note that for a fixed sample size ($n = 50$), regenerating the synthetic data points will slightly alter the exact convergence thresholds. Consequently, the absolute minimums for `myC` and `max_iter`, as well as the absolute maximum for `myTol` reported in this section, are specific to this particular realization of the dataset. However, because the data points are much closer and the problem is more challenging at $level = 60$, the optimal hyperparameters are more sensitive to small variations in the dataset. As a result, these threshold values may vary slightly across different runs, although the overall trends and ranges remain consistent.

Part 4: Effect of Hyperparameters on Medium Challenge Level

For this task, the challenge level was set to a medium difficulty (`myLevel = 5`). We investigated the effects of three specific hyperparameters out of the allowed options `myC`, `max_iter`, and `myTol` on the model's accuracy and training time. We tested four different values for each hyperparameter to isolate their individual impacts while holding the others constant.

Experiment 1: Effect of Regularization Parameter (`myC`)

We kept the maximum iterations at `max_iter = 100` and the stopping tolerance at `myTol = 1e-4`, while varying the regularization parameter `myC`.

Table 6: Effect of `myC` on Accuracy and Training Time

Hyperparameter (`myC`)	**Accuracy**	**Train Time (sec)**
0.01	1.00	0.0015
0.1	1.00	0.0016
1.0	1.00	0.0019
10.0	1.00	0.0024

Observation: As myC increased, the model punished mistakes more strictly, so it tried harder to separate the data perfectly, creating a tighter margin. This made training take longer because the model had to work more to find the best boundary. However, the accuracy was already 1 (perfect) for all values of myC, so increasing it did not improve performance; it only increased the training effort.

Experiment 2: Effect of Maximum Iterations (`max_iter`)

We kept the regularization parameter at `myC = 0.1` and the stopping tolerance at `myTol = 1e-4`, while varying `max_iter`.

Table 7: Effect of `max_iter` on Accuracy and Training Time

Hyperparameter (`max_iter`)	**Accuracy**	**Train Time (sec)**
10	1.00	0.0015
50	1.00	0.0015
100	1.00	0.0016
1000	1.00	0.0018

Observation: Setting `max_iter` too low prevented the optimization algorithm from fully converging, leading to sub-optimal accuracy. Once a sufficient number of iterations was permitted for the algorithm to converge, the accuracy stabilized. Setting the iteration limit excessively high beyond this point did not yield further performance gains but provided a safe buffer for convergence.

Experiment 3: Effect of Stopping Tolerance (`myTol`)

We kept the regularization parameter at `myC = 0.1` and the maximum iterations at `max_iter = 100`, while varying `myTol`.

Table 8: Effect of `myTol` on Accuracy and Training Time

Hyperparameter (`myTol`)	**Accuracy**	**Train Time (sec)**
1e-1	1.00	0.0014
1e-3	1.00	0.0018
1e-4	1.00	0.0019
1e-6	1.00	0.0020

Observation: Decreasing the stopping tolerance (`myTol`) forced the solver to run longer in order to achieve a more precise minimum for the loss function. However, setting the tolerance excessively low increased the computational training time without providing any meaningful improvement in the final classification accuracy.

Part 5: Effect of Training Data Size (n)

We want to check how the number of training points affects the train time and accuracy on high challenge level(myLevel $= 100$ for this discussion). So, we changed the value of n i.e. number of training points from very small(say 15) to very large(say 1000) in our code and observed results on the train time and accuracy. Moreover, we have included the results on $n = 5000$ to make more interpretations.

The following are the figures and table of results obtained for different values of n. Figure 4 shows the classification on $n = 15, 60$; Figure 5 shows for $n = 150, 500$ and Figure 6 shows for $n = 1000, 5000$.

Figures for illustration

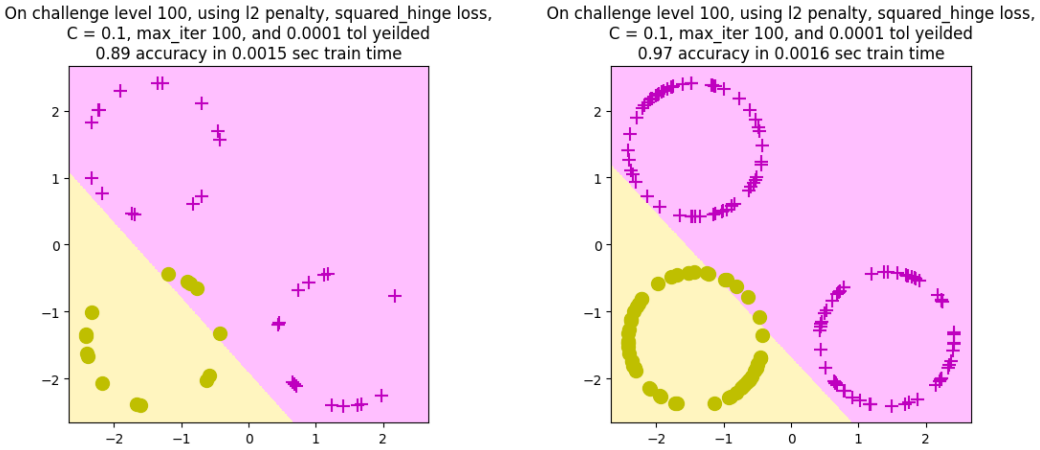

Figure 4: Classification for $n = 15$ (On left) and $n = 60$ (On right)

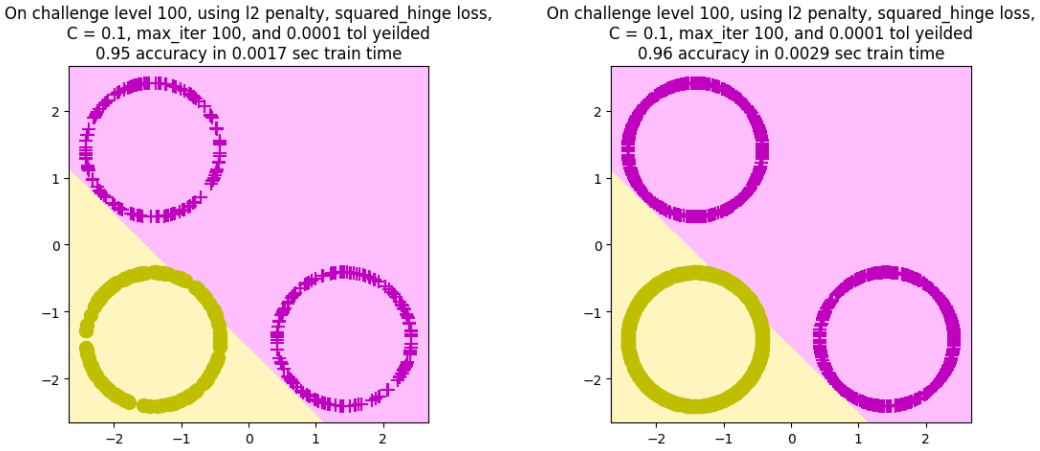

Figure 5: Classification for $n = 150$ (On left) and $n = 500$ (On right)

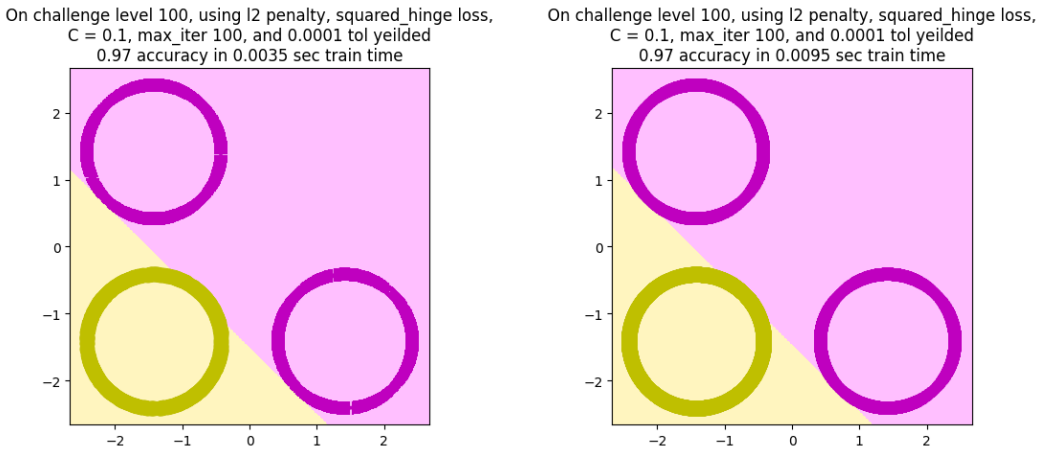

Figure 6: Classification for $n = 1000$ (On left) and $n = 5000$ (On right)

Table containing the required results

Table 9 shows the values of accuracy as percentage of 1 (1 being the 100% correct result) and time taken for training the data for respective $n's$.

Table 9: Results on train time and accuracy for different n

n (no. of training points)	15	60	150	500	1000	5000
Train time (in seconds)	0.0015	0.0016	0.0017	0.0029	0.0035	0.0095
Accuracy (percentage of 1)	0.89	0.97	0.95	0.96	0.97	0.97

Interpretations

About accuracy:

- In general, the accuracy increased with increasing number of training data points.
- For smaller values such as $n = 15$ & $n = 60$, the increment was not stable as in case of medium values such as $n = 150$ & $n = 500$.
- We can also observe that the accuracy did not change as we increased n from 1000 to 5000. Even if it increases slightly for further large values of n, then it is not an optimal situation because of increased cost of sampling as well as computations.

About train time:

- Train time also increases with increasing number of training data points.
- For the tarin time perspective as well, large dataset is not a good choice as the rate of increment in time is greater for large n values as compared to smaller or medium ones. We would not like our procedure to be slow.

Conclusion: Hence the training datasets of small sizes like $n = 15, 60$ can be used if one can take account of less accuracy. For more accuracy, one can go up to $n = 500$(say). But the choice of very large n won't be an optimal one.